# DATA STRUCTURES

# Introduction

# Instructor: Lec Marium Hida

Contact: marium.hida@nu.edu.pk
C513C

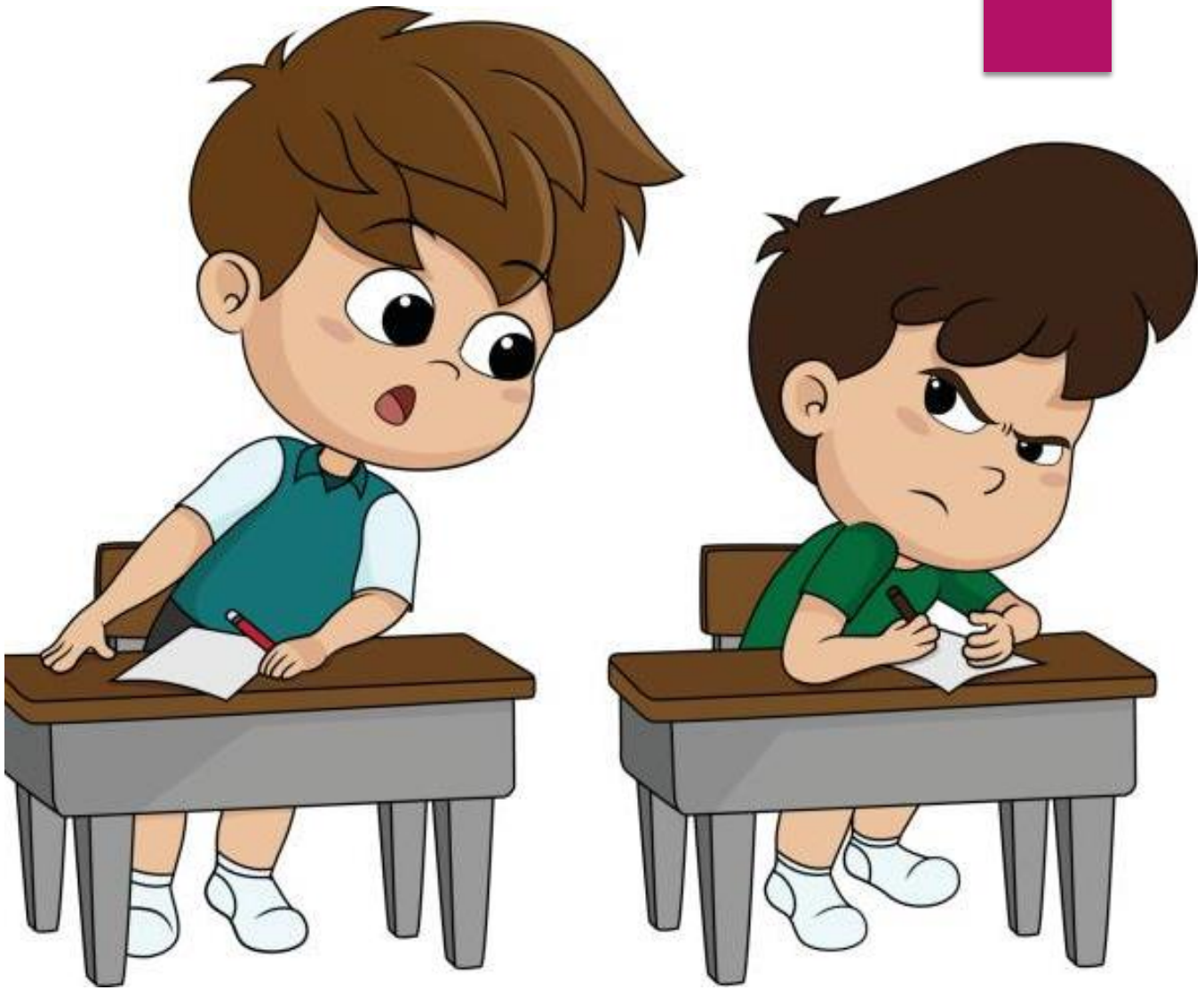
# General Rules & Regulations

- No Late Arrivals.
- No Disturbance In Class
- No Use Of Mobile Phones(keep Them Silent And In Your Pockets)
- No Retake Of Quizzes/Assignments
- No Late Submissions
- Student Is Responsible For His/Her Short Attendance.

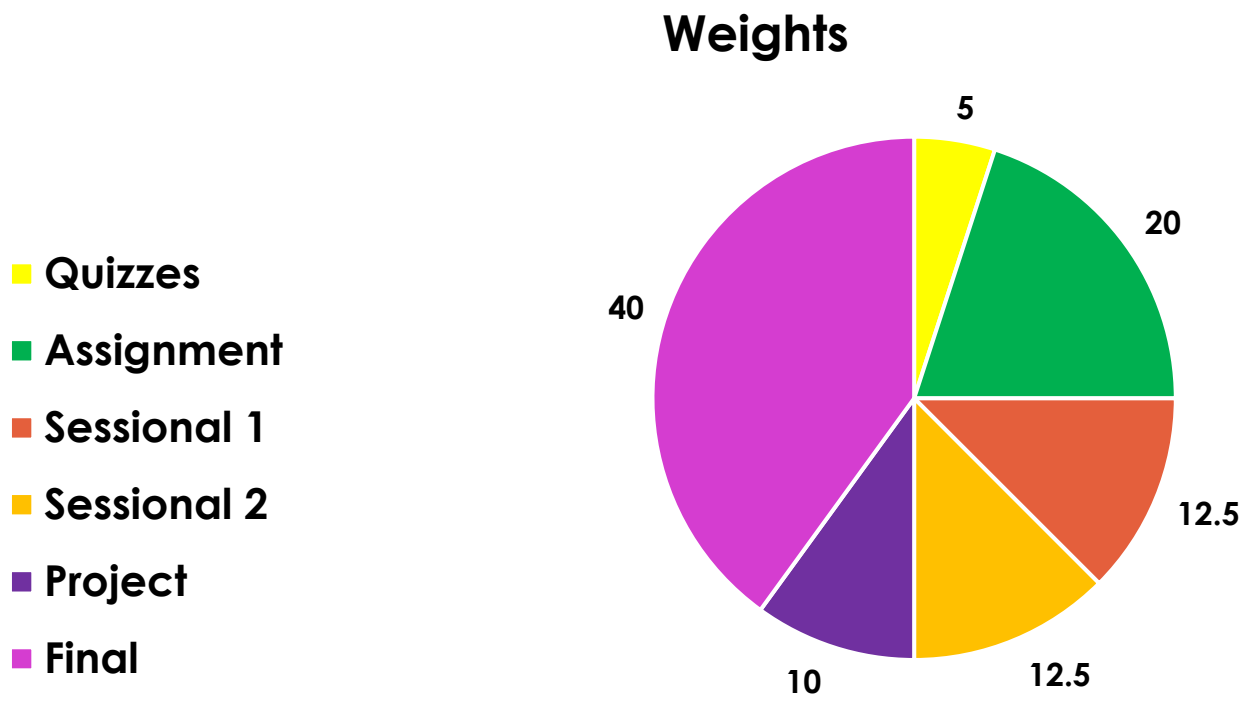
# Dishonesty, Plagiarism

- All parties involved in any kind of cheating in any exam (Quizzes, Assignments & Projects) will get ZERO in that exam

- Habitual cases will be awarded F



# Evaluation Breakdown



# Office Hours

| | |
|---|---|
| **Tuesday** | **8:30 am-1:00pm** |
| **Thursday** | **8:30 am-1:00pm** |

# Course Information

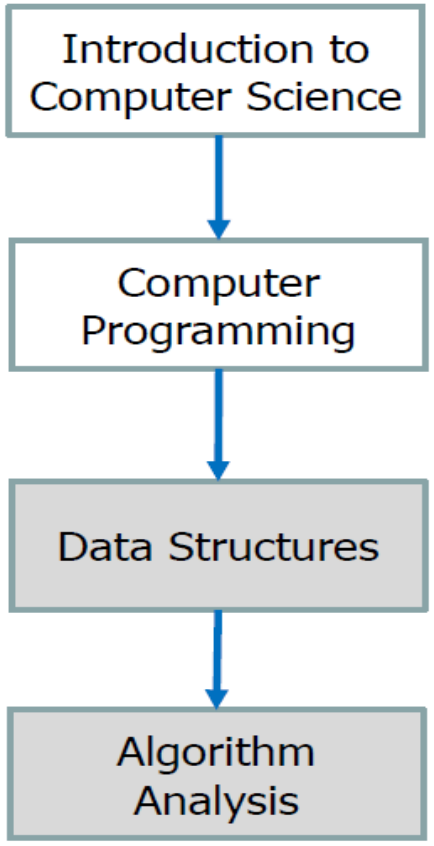
- Pre-requisite
  - Programming Fundamentals/Object Oriented Programming
- Course meeting times
  - Lectures: 2 Sessions/Week
- Course Resource
  - Google Classroom
    - Data Structure Fall-2023
    - shu2ocj

# Course Aim

- To emphasize the importance of data structures and efficient algorithmic design for better management of computing resources while programming.

- In simple words, you will learn **how to write efficient programs**

# General Overview



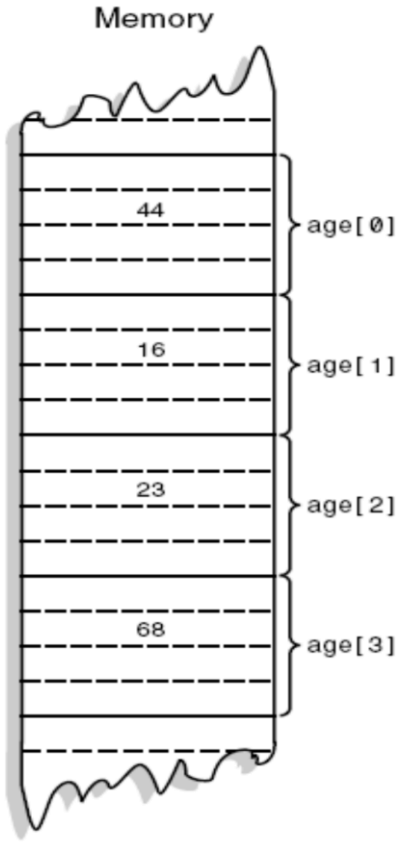

- What is Hardware, Software, Programming, Operating System etc.
- How to write software with the help of procedural and object-oriented programming?
- How to organize information/data, usually in memory, so that it can be used efficiently?
- How to analyze the performance of algorithms to evaluate their suitability for solving complex problems?

# Introduction

- What is data?
  - information converted into binary/digital form
- Types of data?
  - text, numeric, audio, video etc.
- Importance of data?
  - primary purpose of computer programs is to store and retrieve information, usually as fast as possible
- More typically, a data structure provides a way for organizing a collection of data items
- Most data structures have associated algorithms to perform operations on data

# What is a Data Structure?

- A particular way of organizing data in a computer so that it can be used effectively
  - E.g. we can store a list of items having the same data-type using the *array* data structure
- Method of representing logical relationships between individual data elements related to the solution of a given problem
  - Resource constraints
- → space, time



# Costs and Benefits

- Each data structure requires
  - Space for storing data items
  - Time to perform each basic operation
  - Programming efforts
- Cost of a data structure
  - Time and space resources in consumes
- Choice of data structure depends on many factors
  - Type of data
  - Frequency with which various data operations are applied
- Hard to define data structure that performs better in all situations
  - Time and space tradeoff

# Motivational Example

- A cellular service company provides contract to its 10 million users
- Due to new security enforcements, the company wants to prevent issuing of multiple contracts to users

- Method of Detecting Multiple Contracts
  - Before issuing a new contract to user
  - First search the id of user in existing contracts database
  - In case of failure → issue a new contract
  - In case of success → do not issue a new contract to user

# Example: Linear Array Data Structure

- Linear Array (with 10 million entries)
  - 3 arrays (NIC, Name, Address)
  - Structured array
  - Class's object array

| NIC# | Name | Address |
|---|---|---|
| 6584495-9 | Muhammad Faheem | House No 3 Gulshan Bahar Sec 16 |
| 1748425-5 | Naeem Alam | A-11 Shams Plaza Block-B N.Nazimabad |
| 0889679-1 | Arslan Akhtar | H No 152 Bostang Colony |
| 3419668-1 | Zain Ahmed | Sharfabad Street Gulshan Karachi |
| 3445864-3 | Sumair Farooq | Post Office Tayyar, Multan |
| 6395653-4 | Ali Affan | H.No. 425, Sector F-11/4, Islamabad |
| 8224641-1 | Syed Faraz | Sharfabad Street Gulshan, Faisalabad |

Any disadvantage of linear array data structure?
How to improve?

# Example: Improved Data Structure

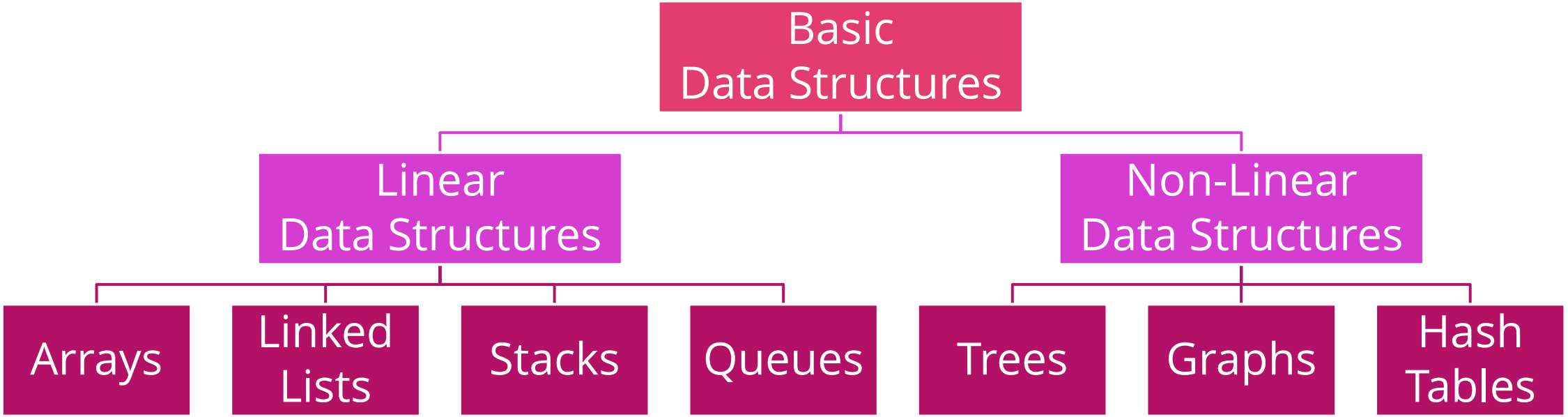
- Create a dictionary data structure
  - Group similar records together
  - Similarity in terms of first digit of NIC number
  - Add a dictionary entry for each distinct digit (0 – 9)

- Example: 3419668-1, 3445864-3, 1748425-5.
  - 3 and 1 are dictionary entries

- Existing contracts are searched in two steps
  1. Search the dictionary entry (i.e., group searching)
  2. Search contract within the group (i.e., with the same NIC digit)

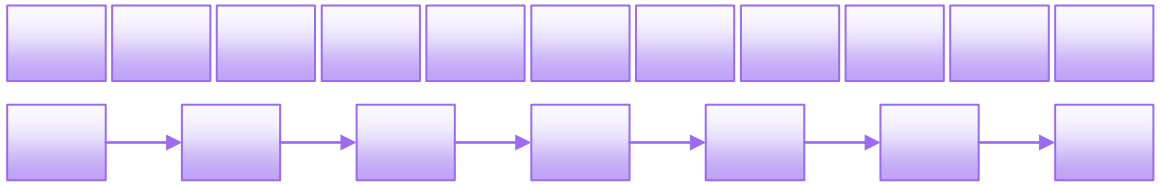
- How much improvement w.r.t. linear array?

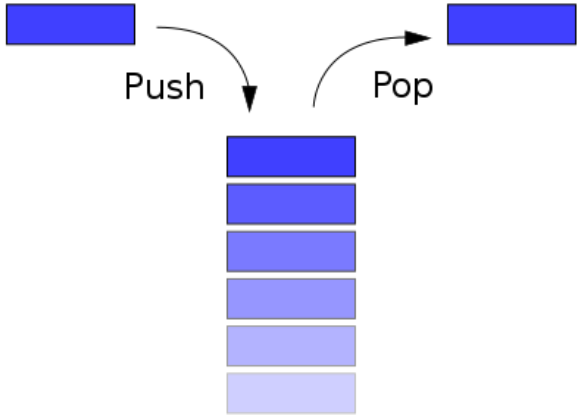
# Basic Data Structures

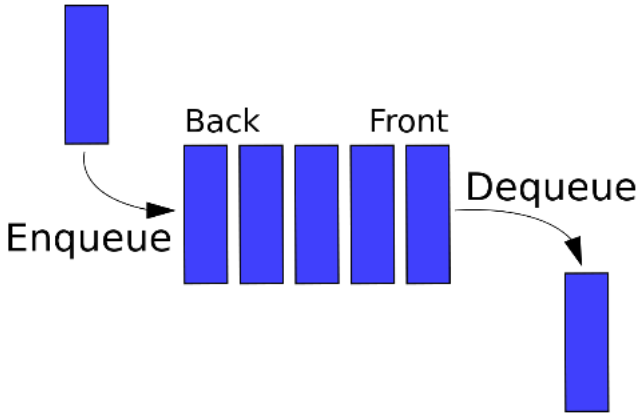


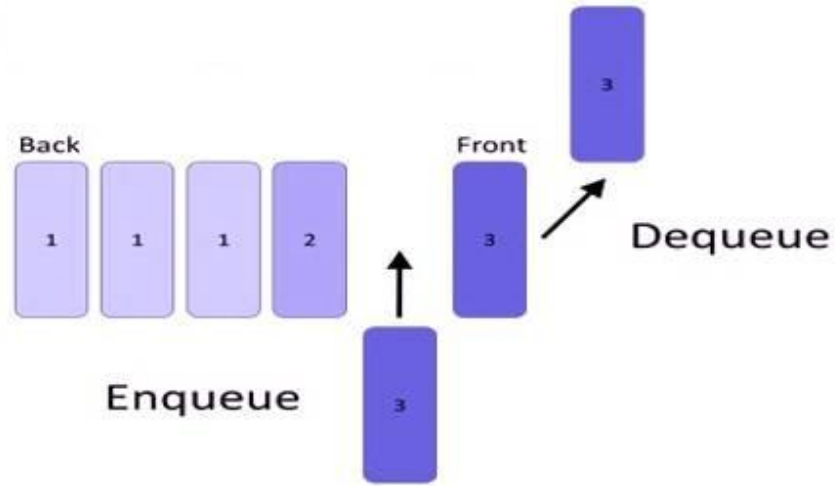
# Basic Data Structures

- Linear: *Values are arranged in a linear fashion*
  - Array: *Fixed size*
  - Linked-List: *Variable size*
  - Stack: *Add to top & remove from top*
  - Queue: *Add to back & remove from front*
  - Priority Queue: *Add anywhere, remove the highest priority*



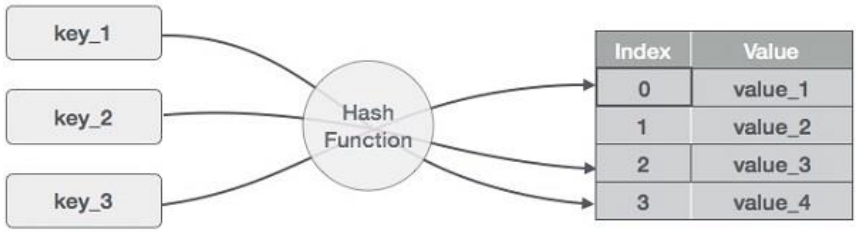


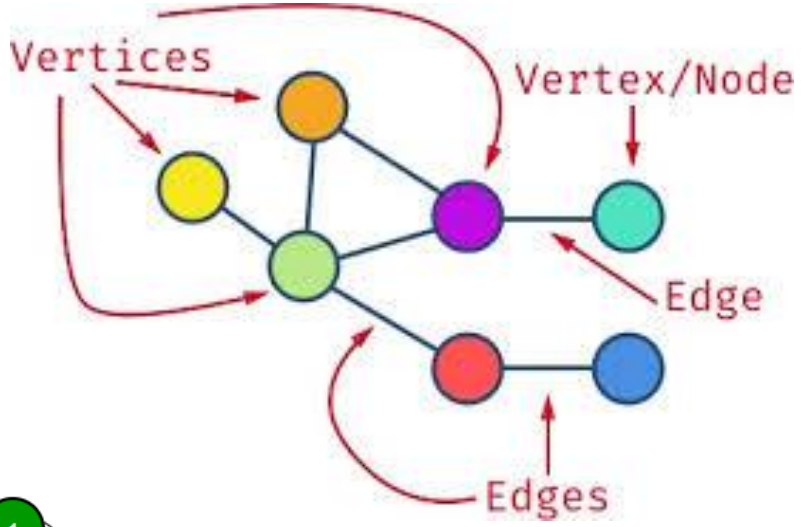



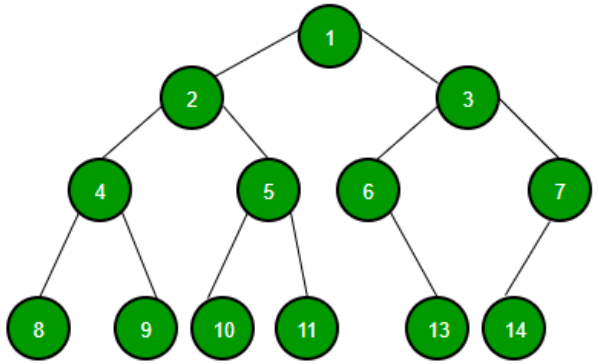



# Basic Data Structures

- Non-Linear: *Values are not arranged in a linear fashion*
  - Hash Table: *Unordered lists which use hash function to insert & search*
  - Tree: *Data is organized in branches*
  - Graph: *A more general branching structure with less strict connection conditions than that for a tree*

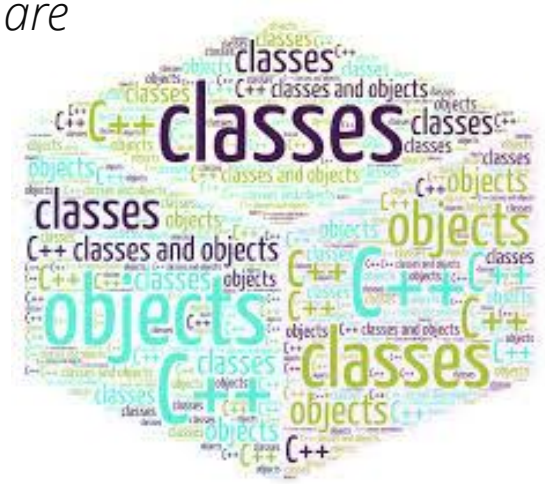


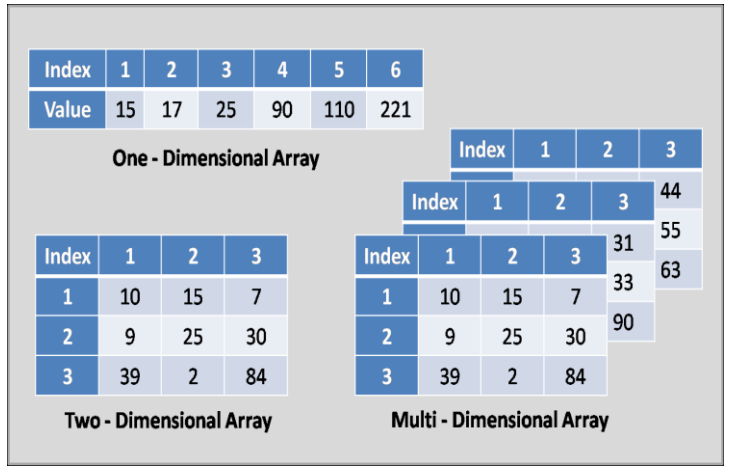



# Types of Data Structures (composition)

- Homogenous: *In this type of data structures, values of the same types of data are stored*
  - Array
- Non-Homogenous: *In this type of data structures, data values of different types are grouped and stored*
  - Structures
  - Classes



# Selection of Data Structure

- Each data structure has costs and benefits
  - Rarely is one data structure better than another in all situations
- A data structure requires:
  - space for each data item it stores
  - time to perform each basic operation
  - programming effort
- Each problem has constraints on available time and space
  - Only after a careful analysis of problem characteristics can we know the best data structure for the task

# Characteristics of Data Structures

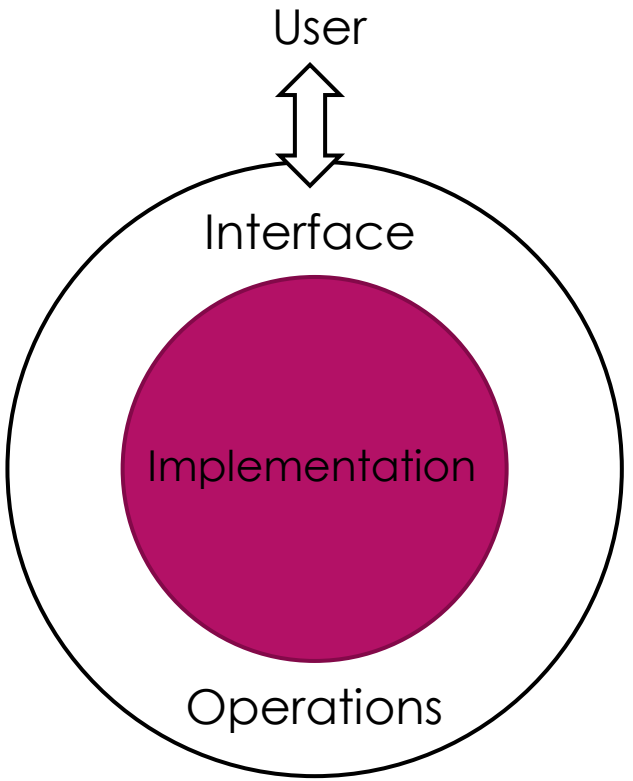
| Data Structure | Advantages | Disadvantages |
|---|---|---|
| **Array** | Quick inserts<br>Fast access if index known | Slow search<br>Slow deletes<br>Fixed size |
| **Ordered Array** | Faster search than unsorted array | Slow inserts<br>Slow deletes<br>Fixed size |
| **Stack** | Last-in, first-out acces | Slow access to other items |
| **Queue** | First-in, first-out access | Slow access to other items |
| **Linked List** | Quick inserts<br>Quick deletes | Slow search |
| **Binary Tree** | Quick search<br>Quick inserts<br>Quick deletes<br>*(If the tree remains balanced)* | Deletion algorithm is complex |
| **Red-Black Tree** | Quick search<br>Quick inserts<br>Quick deletes<br>*(Tree always remains balanced)* | Complex to implement |

# Characteristics of Data Structures

| Data Structure | Advantages | Disadvantages |
|---|---|---|
| **2-3-4 Tree** | Quick search<br>Quick inserts<br>Quick deletes<br>*(Tree always remains balanced)*<br>*(Similar trees good for disk storage)* | Complex to implement |
| **Hash Table** | Very fast access if key is known<br>Quick inserts | Slow deletes<br>Access slow if key is not known<br>Inefficient memory usage |
| **Heap** | Quick inserts<br>Quick deletes<br>Access to largest item | Slow access to other items |
| **Graph** | Best models real-world situations | Some algorithms are slow and very complex |

# Abstract Data Types (1)

- A definition of data type solely in terms of
  - Set of related data items (or values)
  - Set of operations on the data
- Separation of logical properties from the implementation details
  - Hide implementation details (**Encapsulation!**)
- What not how is focus



# Abstract Data Types – Examples

- Whole numbers (integers)
  - Operations
    - arithmetic operations (addition, subtraction, etc.)
- Flight reservation
  - List of seats
  - Operations
    - Find empty seat
    - Reserve a seat
    - Cancel a seat assignment

# Abstract Data Types (2)

- ADTs definition consists of
  - Storage structures (i.e., data structures) to store data items
  - Algorithms for basic operations

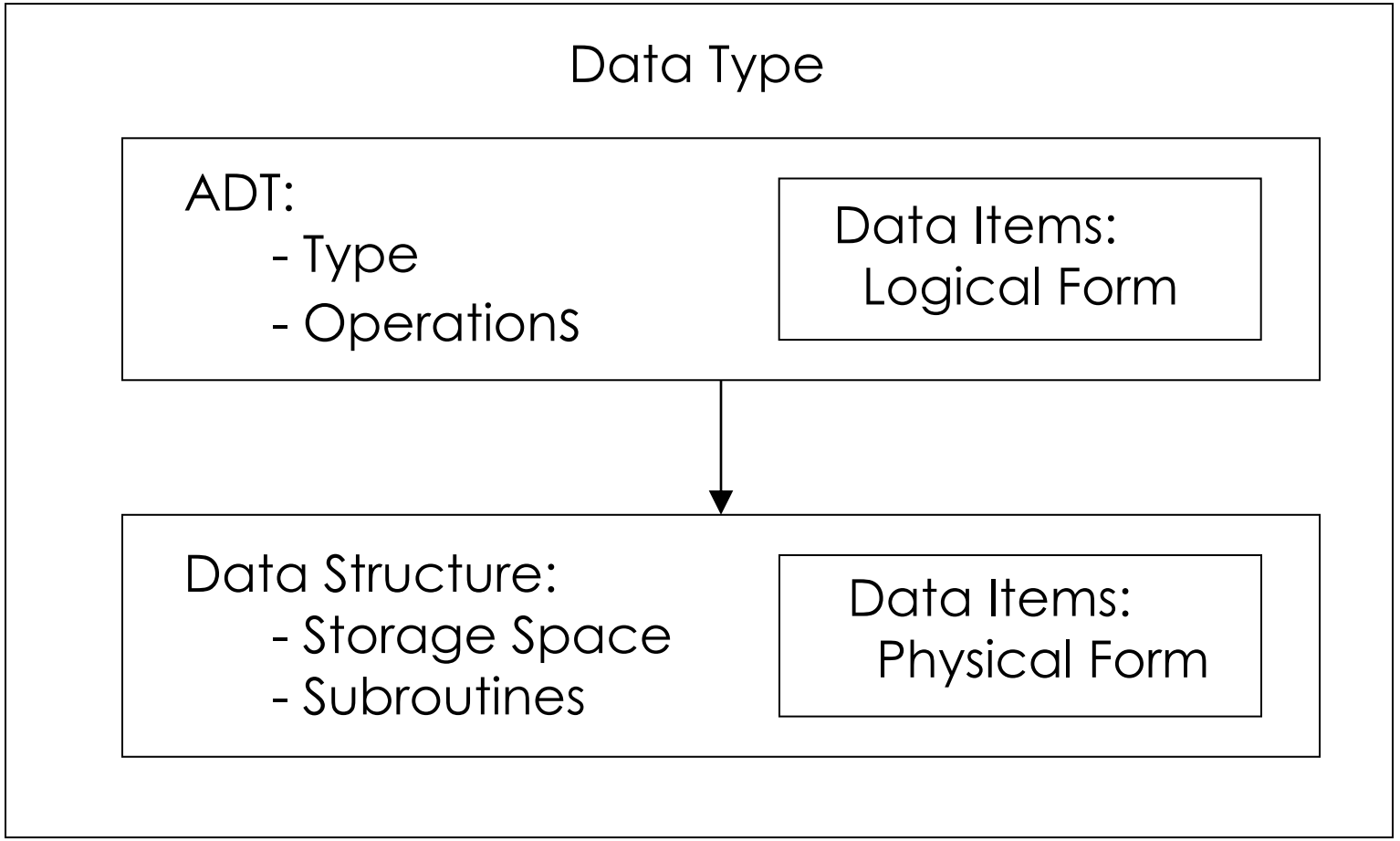
- Storage structures/data structures used in the implementation
  - Provided in a language (primitive or build-in)
  - Built from the language constructs (user-defined)

→Separation of a data type from its implementation

# Data Structures

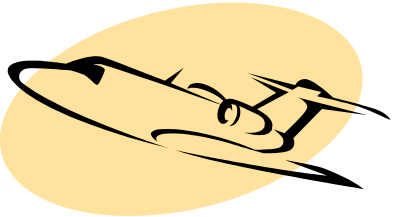
- A data structure is physical implementation of an ADT
  - Each operation associated with ADT is implemented by one or more subroutines in the implementation
- Data structure usually refers to an organization of data in main memory
- File structure is an organization of data on peripheral storage such as disk drive

# ADT vs. Data Structures



# Example: Airplane Flight Reservation (1)

- Consider example of an airplane flight with 10 seats to be assigned



- Operations
  - List available seats
  - Reserve a seat

- Implementation: How to store, access data?
  - 10 individual variables

# Implementation: 10 Individual Variables

List available seats:

```
1. if seat1 == ‘ ’;
    display 1
2. if seat2 == ‘ ’;
     display 2
.
.
.

10. if seat10 == ‘ ’;
     display 10
```

Reserve a seat:

```
1. Set DONE to false
2. if seat1 ==‘ ’;
   print “do you want seat #1??”
   Get answer
   if answer==‘Y’;
    set seat1 to ‘X’
    set Done to True
3. if seat2 ==‘ ’ and Done == false;
   print “do you want seat #2??”
   Get answer
   if answer==‘Y’;
    set seat2 to ‘X’
    set Done to True
```

# Example: Airplane Flight Reservation (2)

- Consider example of an airplane flight with 10 seats to be assigned



- Operations
  - List available seats
  - Reserve a seat

- Implementation: How to store, access data?
  - 10 individual variables
  - An array of variables

# Implementation: An array of variables

List available seats:

```
for number ranging from 0 to max_seats-1, do:
    if seat[number] == ‘ ’;
        Display number
```

Reserve a seat:

```
Reading number of seat to be reserved
if seat[number] is equal to ‘ ’;
    set seat[number] to ‘X’
else
    Display a message that the seat having this number is
    occupied
```

# Example: Airplane Flight Reservation (2)

- This simple example illustrate the concept of an Abstract Data Type
- ADT consists of
  - Collection of data items
  - Basic operations that must be performed on them
- In the example, a collection of data is a list of seats
- Basic operations are
  - List available seats
  - Reserve a seat

# ADTs for Primitive Data Types

# Boolean Data

- Data values: {false, true}
- Operations:
  - And &&
  - Or ||
  - Not !

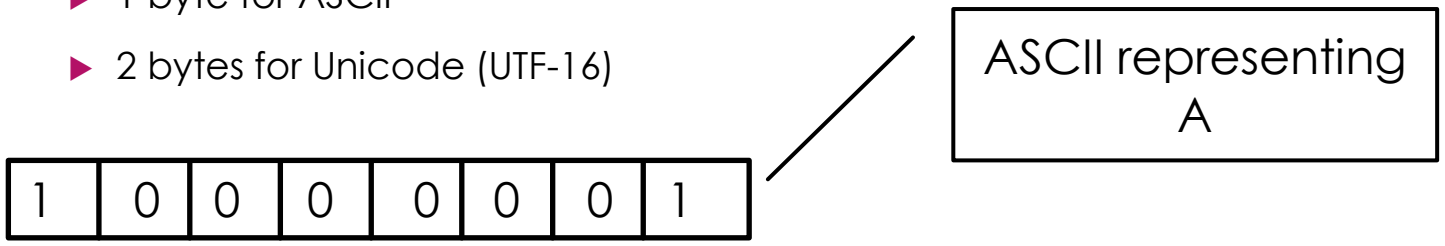
| && | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

| \|\| | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 1 |

| $x$ | $!x$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

# Character Data

- Data values
  - Numeric codes (ASCII, Unicode)
    - 1 byte for ASCII
    - 2 bytes for Unicode (UTF-16)




- Basic Operations
  - Comparison (equal, less than, greater)
  - Use of numeric codes
  - ....

# Integer Data

- Data values
  - Non-negative (unsinged) integer
    - Base-two representation in a fixed number of bits (e.g., 32 bits )
    - 88 : 0000 0000 0000 0000 0000 0000 0101 1000
  - Signed integer
    - Ones' or Two's complement representation in fixed number of bits
    - 127 : 01111111, -127: 10000000 (Ones' complement)
    - 127: 01111111, -127: 10000001 (Two's complement)
- Operations
  - Arithmetic operations
    - Addition, subtraction, multiplication, division

# Data Modeling

- Data Model is an abstract model that organizes elements of data and standardizes how they relate to one another and to the properties of real-world entities
  - explicitly determines the structure of data and therefore referred to as data structures in the context of programming languages
- The choice of particular data model depends on two consideration:
  - It must be rich enough in structure to represent the relationship between data elements
  - The structure should be simple enough that one can effectively process the data when necessary

# Abstract Data Type (ADT)

## Abstract Data Type

- *ADT* stores data and allow various operations on the data to access and change it
- A mathematical model, together with various operations defined on the model
- An ADT is a collection of data and associated operations for manipulating that data

## Data Structures

- Physical implementation of an ADT
- Data structures used in implementations are provided in a language *(primitive* or *built-in)* or are built from the language constructs *(user-defined)*
- Each operation associated with the ADT is implemented by one or more subroutines in the implementation

# Abstract Data Type (ADT)

- ADT is a way of looking at a data structure focusing on what it does and ignoring how it does its job
  - E.g., A **stack** is an abstract data type supporting *push*, *pop* and *isEmpty* operations
  - It can be implemented using an array or using a linked-list
- ADTs support *abstraction*, *encapsulation*, and *information hiding*
  - *Abstraction* is the structuring of a problem into well-defined entities by defining their data and operations
  - The principle of hiding the used data structure and to only provide a well-defined interface is known as *encapsulation*

# Abstract Data Type (ADT)

- The core operations of ADT
  - Every Collection ADT should provide a way to:
    - add an item
    - remove an item
    - find, retrieve, or access an item
  - Many, many more possibilities
    - is the collection empty
    - make the collection empty
    - give me a sub-set of the collection

# Review of Arrays & Pointers in C++

# Review of Arrays & Pointers in C++

- Every byte in the computer's memory has an address
  - Addresses are numbers just as every house in a locality has got an address
- Your program, when it is loaded into memory, occupies certain range of these addresses
  - This means that every variable and every function in your program starts with a particular address

- **NOTE : Pointer can only contain the ADDRESS of certain memory location**

# Review of Arrays & Pointers in C++

- Normally a variable directly contains a specific value
- Pointer, on the other hand, contains an address of the variable that contains the specific value
  - Pointers, like other variables must be declared before they are used

```
int *ptr, temp;
```

  - Pointers should be initialized either when they are declared or in an assignment statement
  - A pointer may be initialized to `0`, `NULL` or an address
    - A pointer with the value `0` **or** `NULL` points to nothing

# Review of Arrays & Pointers in C++

- `NULL` is a symbolic constant defined in the header file `<iostream.h>`
  - Initializing a pointer to `NULL` is equivalent to initializing a pointer to `0`
  - Initializing a pointer is necessary to prevent it pointing to unknown or un-initialized area of memory
- There are two pointer operators
  - the *address of* operator `&`
  - *indirection operator* or *dereferencing operator* `*`

# Review of Arrays & Pointers in C++

- `&` is a unary operator that returns **the *address*** *of* its operand

```
int x = 5;
int *ptrr = NULL;
ptrr  = &x;
```

- In above `*` shows **`ptrr`** is pointer variable whose type is **`int`**
  - i.e. it can only contain the *address of* any *integer variables* in memory

# Review of Arrays & Pointers in C++

```
cout<< *ptrr << "\n" ;
cout<<  x    << "\n" ;
```

- Will produce the same result 5
- Here * means *value of* the variable to which `ptrr` points to, which is `x`

- Note that the dereferenced pointer may also be used on the left side of an assignment statement as follows.

```
*pttr = 9;       //or
cin>>*ptrr;
```

```
#include<iostream.h>
void main( )
{
      int x = 5;
      int *ptrr = NULL;
      ptrr  = &x;
      cout<<  *ptrr <<"\n";
      cout<<  x  <<"\n";
      *pttr = 9;
      cout<< &x <<"\n";
      cout<< *&ptrr <<"\n";      // cancel effect of each other
      cout<< &*ptrr <<"\n";
      cout<< ptrr <<"\n";
      cout<< *ptrr <<"\n";
      cout<< x <<"\n";
}
```

**Output of Program**

```
5
5
0x0064FDF4
0x0064FDF4
0x0064FDF4
0x0064FDF4
9
9
```

# `new` and `delete` operators

- Variables created during execution (run time) of a program by means of special operation is known as *Dynamic Data*
- In C++ operation is `new`

```
int *intptr;
char *namestr;
intptr = new int;
namestr = new char[8];
```

- Variables created by **`new`** are said to be on the *free store* or *heap*, a region of memory set aside for *Dynamic Variables*
- The **`new`** operator obtains a chunk (portion) of memory from the *free store*

# `new` and `delete` operators

- *Dynamic data* can be destroyed at any time during the execution of a program when it is no longer needed
  - The built-in operator delete does that and has two forms
    - one for deleting single variable
    - the other for deleting an array

```
delete pointer;
delete [ ] pointer;
```

```cpp
#include<iostream>
using namespace std;

int main() {
    int x, n;
    cout << "Enter the number of items:" << "\n";
    cin >>n;
    int *arr = new int(n);
    cout << "Enter " << n << " items" << endl;
    for (x = 0; x < n; x++) {
        cin >> arr[x];
    }
    cout << "You entered: ";
    for (x = 0; x < n; x++) {
        cout << arr[x] << " ";
    }
    return 0;
}
```